

Shahid Khan

315-746-1247 | shahidhkhan.dev@gmail.com | LinkedIn: [shahid-h-khan](#) | GitHub: [ShahidHKhan](#)

EDUCATION

State University of New York, New Paltz

Bachelor of Science in Computer Science – Cumulative GPA: 3.4/4.0

New Paltz, NY

Aug. 2023 – May 2027

Certifications: Google Cloud Generative AI Leader (2025), AI Engineer Core Track (2026)

Awards: 2026 New Paltz Hackathon – 1st Place

Leadership: Muslim Student Association President (2025), Military Affiliated Club Vice President (2024)

PROFESSIONAL EXPERIENCE

Information Technology Technician – SUNY New Paltz

Aug. 2025 – Present

- Resolved ~30 tickets/week for 100+ users by troubleshooting hardware and software issues in **TeamDynamix**.
- Secured campus user access by managing accounts, permissions, and password resets in **Microsoft Azure**.
- Prepared new laptops and devices for day-one use by imaging, configuring, and deploying them to faculty and staff.
- Restored Wi-Fi, printer, and classroom AV service by diagnosing connectivity and hardware issues on-site.

AI Engineer Intern – SUNY New Paltz

Jun. 2026 – Aug. 2026

- Cut resolution time on difficult tickets by **30%** by designing and deploying “HawkEye,” an LLM-powered RAG assistant.
- Halved schedule generation time (**4 weeks to 2**) by building “Desk-Scheduler,” an LLM and constraint-solver pipeline.
- Improved HawkEye’s retrieval quality by **25%** by benchmarking two embedding designs and shipping the stronger one.
- Drove adoption in a **20-person IT office (2–3 uses/day)** by tracking usage, applying feedback, and cutting latency/cost.

AI Quality Engineer Intern – GridDevs

Sept. 2025 – Dec. 2025

- Identified the most reliable retrieval configuration for a RAG system, as measured by **MRR, nDCG, and keyword coverage**, by benchmarking multiple embedding models and LLMs against each other.
- Assessed LangChain-based versus native code implementations of the RAG pipeline, evaluating each against performance, stability, and maintainability criteria to inform quality decisions.
- Built automated Gradle test suites to validate data workflows and catch regressions across pipeline iterations.

PROJECTS

HawkEye – Python, LangChain, Gradio, PostgreSQL/pgvector, Supabase

github.com/ShahidHKhan/HawkEye

- Improved retrieval accuracy on a **100-question evaluation set** (measured by **MRR and nDCG**) over an **850+ article** knowledge base by building a multi-stage pipeline with query expansion, source merging, and LLM reranking.
- Migrated the vector store from ChromaDB to Postgres/pgvector on Supabase for production-scale storage, writing a backfill script to safely re-embed **4,000+ chunks** with duplicate-run protection.
- Kept the knowledge base current automatically by building a weekly **GitHub Actions** pipeline that re-embeds only new or changed articles, with a circuit breaker that aborts suspicious mass deletions.

AutoPricer – Python, PyTorch, XGBoost, FastAPI, Modal

github.com/ShahidHKhan/AutoPricer

- Built an ML pricing system on **500K used-car listings**, reducing baseline MAE by **71%** (\$9,781 to \$2,799) with a tuned XGBoost model, and engineered a multi-agent ensemble (QLoRA fine-tuned Llama-3.2-3B + RAG-augmented Gemini over **300K ChromaDB embeddings**) that further reduced MAE by **25%** (\$1,599 to \$1,202).
- Cut the Llama-3.2-3B model’s pricing error by **63%** (MAE \$6,723 to \$2,471) by QLoRA fine-tuning it on a free Colab T4 GPU (4-bit NF4, **18M-parameter** LoRA adapter).
- Deployed the inference pipeline as a streaming FastAPI service on Modal (T4 GPU inference), integrated with a Chrome extension that processes live marketplace data to flag mispriced cars in real time.

Desk-Scheduler – Python, LangGraph, Gemini, OR-Tools, Streamlit, SQLite, Docker

github.com/ShahidHKhan/scheduler

- Architected a **LangGraph** workflow using the **Gemini API** for multimodal ingestion, extracting structured availability data from unstructured PDFs, spreadsheets, and photos.
- Formulated a deterministic constraint model with **Google OR-Tools (CP-SAT)** to enforce scheduling rules, separating LLM parsing from provable assignment logic to prevent hallucinated schedules.
- Guaranteed all **6 hard scheduling rules** hold, verified by a **16-test pytest suite** of real solves, and added infeasibility diagnosis with an **LLM-as-a-judge** that fact-checks explanations against solver data.

SKILLS

LLM Engineering: RAG, Reranking, LangChain, LangGraph, Agents/Tool Calling, Pydantic, LLM-as-a-Judge Evals

Machine Learning: PyTorch, Hugging Face (TRL, QLoRA), scikit-learn, XGBoost, Weights & Biases, Google OR-Tools

Backend & Frameworks: Python, TypeScript/JavaScript, FastAPI, Node.js, Streamlit, Gradio, REST API Development

Databases & Deployment: PostgreSQL (pgvector), Supabase, SQLite, ChromaDB, Modal, Docker, Git, Azure